

Alexander Read

Introduction

We consider the positive implicational fragment of the intuitionistic propositional calculus, a language $\mathcal{L}^\rightarrow$. The alphabet consists of $PV \equiv \{p_0, p_1, p_2, \dots\}$, an infinite set of propositional variables, and a binary connective $\rightarrow$. The well-formed formulae ('wff') are defined by the following grammar, where ϕ, ψ are meta-variables ranging over formulae:¹

$$\phi, \psi ::= PV \mid (\phi \rightarrow \psi)$$

I've defined PV in this way because it closely mirrors my implementation in Haskell. However, in this post I'll use ' p ', ' q ', and ' r ' as variables to match the usual notation.

Anyway, following Łukasiewicz, Meredith, Prior, and others (e.g., see [here](#)), a lot of the papers I've been reading on $\mathcal{L}^\rightarrow$ use prefix, rather than infix, notation to write formulae. For example, where ' C ' is the prefix-style implication operator, you often see formulae written using the style on the left, not that on the right:

$$CCpCqrCCpqCpr \equiv (p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r)) \quad (\text{S})$$

$$CpCqp \equiv (p \rightarrow (q \rightarrow p)). \quad (\text{K})$$

I hope you agree that the prefix notation is pretty awkward to read.²

So, I wanted a program to convert from prefix to infix notation. This is done by parsing valid prefix strings into ASTs, and then printing those trees in infix format (i.e., by in-order traversal). Rather than use an existing library like [Parsec](#), this was a nice opportunity to implement a simple parser by hand, following the tutorials from Hutton and Meijer ([1996](#), [1998](#)).

-
1. Do we define parentheses as part of the vocabulary? Studd does, but Leader notes that they're not strictly part of the language, just used to disambiguate the ASTs. See also the *Unique Readability Theorem* in the B1 §2 notes. We can also use these notes and the Problem Set 1 to define Polish notation and relate it to the prefix notation.
 2. The examples correspond to the types of the combinators **S** and **K** in combinatory logic.